

UNIVERSIDAD DE ANTIOQUIA

FACULTAD DE INGENIERIA

PROFESOR: ANÍBAL JOSE GUERRA

INTEGRANTES: MIGUEL ANGEL UPEGUI LONDOÑO. DAVID ALEJANDRO MARÍN PINZÓN.

INFORME PRELIMINAR. ESTRUCTURA DE DATOS, ANÁLISIS DEL PROBLEMA, Y DISEÑO.

CONTEXTUALIZACIÓN.

El desafío consiste en diseñar e implementar una versión del juego Sweet Crush en C++, utilizando una representación compacta de las fichas mediante tres bits por posición. Además de las funciones básicas de un juego de combinaciones, el programa debe permitir modificar dinámicamente las dimensiones del tablero y administrar la memoria utilizada. La propuesta se desarrollará de manera modular y deberá cumplir las restricciones de programación establecidas en el enunciado.

ESTRUCTURA DE DATOS.

La estructura de datos que se va a usar para este desafío será un arreglo dinámico unidimensional de tipo `unsigned char*`, organizado como una secuencia continua de bytes. Cada byte contiene 8 bits y será utilizado únicamente como almacenamiento físico de la secuencia compacta del tablero. Las fichas no ocuparán un byte completo, sino exactamente 3 bits cada una, por lo que varias fichas podrán compartir un mismo byte.

La posición de cada ficha se determinará a partir de su fila y columna. Primero se obtiene el índice de la posición mediante $\text{fila} * \text{columnas} + \text{columna}$ y luego se calcula el bit inicial multiplicando este índice por 3. A partir de esta posición se determinarán el byte y los bits correspondientes. Esto permitirá trabajar directamente sobre la memoria mediante operadores a nivel de bits, sin crear una copia del tablero en la que cada ficha ocupe un byte completo.

ANÁLISIS.

Después de analizar el enunciado se identificaron varias restricciones importantes, entre ellas la prohibición de strings, STL, struct, class y otras estructuras dinámicas suministradas por bibliotecas. Estas restricciones hacen necesario controlar directamente el tamaño del tablero y la memoria utilizada, ya que las dimensiones pueden cambiar al agregar o eliminar filas y columnas. Se decidió evitar variables globales para el estado principal del tablero y pasar la información necesaria como parámetros a las funciones, con el fin de facilitar la división del programa en varios archivos .h y .cpp y mantener una estructura modular.

La representación principal no será un arreglo donde cada posición corresponda directamente a una ficha. El tablero se manejará como una secuencia compacta de bits, y las funciones de acceso serán las encargadas de localizar los tres bits de cada posición. De esta manera se mantiene la representación exigida por el desafío incluso cuando una ficha quede repartida entre dos bytes consecutivos.

La elección de unsigned char* permite trabajar con bloques de bytes y acceder a cada uno de sus 8 bits mediante operaciones bitwise. La cantidad de memoria no se reservará de manera sobredimensionada, sino que se calculará según las dimensiones actuales del tablero. Para F filas y C columnas se requieren $3 \cdot F \cdot C$ bits y, por tanto, la cantidad mínima de bytes será el redondeo hacia arriba de esa cantidad dividido entre 8.

Para dividir el problema en partes manejables se definieron seis grupos principales: almacenamiento y manipulación de las fichas, visualización del tablero, detección de combinaciones, modificación estructural, procesamiento de cascadas y manejo del estado actual del juego. Cada grupo tendrá funciones específicas que se comunicarán entre sí.

Nuevafila

Borrarfila

Nuevacolumna

Borrarcolumna

Las funciones de borrar fila y columna primero preguntaran que posición del tablero se quiere trabajar, las otras se ejecutaran automáticamente al requerirse en el código y contexto del juego.

Para mantener la representación de tres bits se utilizará el puntero unsigned char* para la memoria dinámica del tablero. Las fichas no se almacenarán como variables independientes; su valor se obtendrá y modificará directamente dentro de la secuencia de bits mediante máscaras y desplazamientos.

Las estadísticas tales como:

- Dimensiones actuales del tablero.
- Cantidad de eliminaciones realizadas por el usuario.
- Cantidad de fichas eliminadas en total.
- Cantidad de combinaciones detectadas.

DISEÑO.

La solución se dividirá en funciones con responsabilidades específicas. La intención es evitar que una sola función concentre toda la lógica del programa y facilitar las pruebas de cada parte durante la implementación.

mostrarEstadoJuego(): muestra las dimensiones actuales, las eliminaciones realizadas, las fichas eliminadas, las combinaciones, las cascadas y la puntuación.

calcularPuntuacion(): calcula la puntuación de acuerdo con el criterio definido para el juego.

Estado del juego

actualizarEstadisticas(): actualiza los contadores de eliminaciones, fichas eliminadas, combinaciones y cascadas producidas.

borrarColumna(): elimina la columna indicada y reorganiza el tablero y la memoria cuando corresponda.

nuevaColumna(): agrega una columna en la posición indicada y reorganiza las posiciones posteriores.

borrarFila(): elimina la fila indicada y conserva las fichas que deben permanecer en el tablero.

Modificación estructural

nuevaFila(): agrega una fila en la posición indicada y reorganiza la información del tablero manteniendo la representación compacta.

procesarCascadas(): vuelve a detectar y procesar combinaciones después de una eliminación, reorganización o incorporación de fichas, hasta que no aparezcan nuevas combinaciones.

Reorganización y cascadas

reorganizarTablero(): desplaza verticalmente las fichas para ocupar los espacios vacíos producidos por las eliminaciones.

eliminarCombinaciones(): elimina las fichas pertenecientes a las combinaciones detectadas, incluyendo las que participan simultáneamente en una combinación horizontal y una vertical.

detectarVertical(): recorre el tablero para localizar grupos de tres o más fichas iguales consecutivas en una columna.

Detección y eliminación de combinaciones

detectarHorizontal(): recorre el tablero para localizar grupos de tres o más fichas iguales consecutivas en una fila.

mostrarTableroBinario(): muestra la secuencia binaria almacenada en memoria para comprobar la representación compacta del tablero.

mostrarTablero(): muestra el tablero de forma comprensible para el jugador utilizando los caracteres definidos para cada tipo de ficha.

llenarTablero(): genera las fichas necesarias para llenar las posiciones del tablero, tanto en su creación inicial como después de una reorganización.

Generación y visualización

generarFicha(): genera aleatoriamente uno de los seis tipos de fichas con distribución uniforme.

modificarFicha(): cambia los tres bits de una posición determinada utilizando máscaras y operadores bitwise, sin alterar las demás posiciones.

Manipulación de fichas

obtenerFicha(): recibe una fila y una columna, calcula la posición de los tres bits y extrae el valor de la ficha mediante operaciones bitwise. También contempla el caso en que los tres bits estén repartidos entre dos bytes.

eliminarFicha(): recibe la posición indicada por el usuario y modifica los tres bits correspondientes para establecer la posición como vacía.

redimensionarMemoria(): ajusta la cantidad de memoria utilizada cuando una modificación del tablero haga necesario realizar una nueva reserva, teniendo en cuenta la condición del 65 %.

liberarTablero(): libera la memoria dinámica utilizada por el tablero.

crearTablero(): reserva dinámicamente la memoria necesaria y establece las dimensiones iniciales del tablero.

Gestión de memoria

calcularBytes(): calcula la cantidad mínima de bytes necesaria para almacenar el tablero según sus filas y columnas.

FLUJO GENERAL DEL PROGRAMA.

Al iniciar, se solicitan las dimensiones del tablero, se calcula la memoria requerida y se generan las fichas iniciales. Durante la partida se muestra el tablero y se recibe la posición seleccionada por el usuario. Luego se elimina la ficha, se buscan combinaciones horizontales y verticales y se procesan las eliminaciones. Después se reorganiza el tablero, se generan nuevas fichas y se comprueba si se produjo una cascada. Finalmente se actualizan las estadísticas y se muestran nuevamente el tablero y su representación binaria. Las operaciones de agregar o eliminar filas y columnas seguirán este mismo principio, modificando las dimensiones y la memoria de acuerdo con los requisitos del desafío.

- Cantidad de cascadas producidas por la eliminación actual.
- Puntuación obtenida.

Las cantidades correspondientes se manejarán mediante contadores y se actualizarán durante la ejecución del juego.